



Vad fan ska vi handla

From prototype to an app you can use

A guide to the backend, services and hosting your shopping list app needs, and how to put them together.

October 2026

Contents

1. Where the app is today	3
2. The big picture	4
3. What the backend has to do	5
4. Choosing a backend	6
5. Setting up Supabase and connecting the app	7
6. Push notifications	11
7. Getting the app onto your phones	12
8. Hosting: where things run	14
9. What it costs	15
10. Step-by-step roadmap	16
11. Security checklist	17
12. Glossary	18

1. Where the app is today

The app already works end to end: you can log in, switch between lists, add items with amount and category, check them off, manage lists, change language and appearance, and get notifications. But everything behind the screens is **pretend**. It runs against a mock backend that lives inside the app itself.

Part	Status
Screens, components, theme	Done. Real code you keep.
Contexts and hooks (AuthContext, ListsContext, useItems ...)	Done. Real code you keep.
api/ folder	The seam: today each file re-exports the mock. This is what you rewrite.
mock/ folder	Fake database, fake network delay, fake second user. Deleted at the end.

Because the mock data lives in the app's memory, three things don't work for real yet:

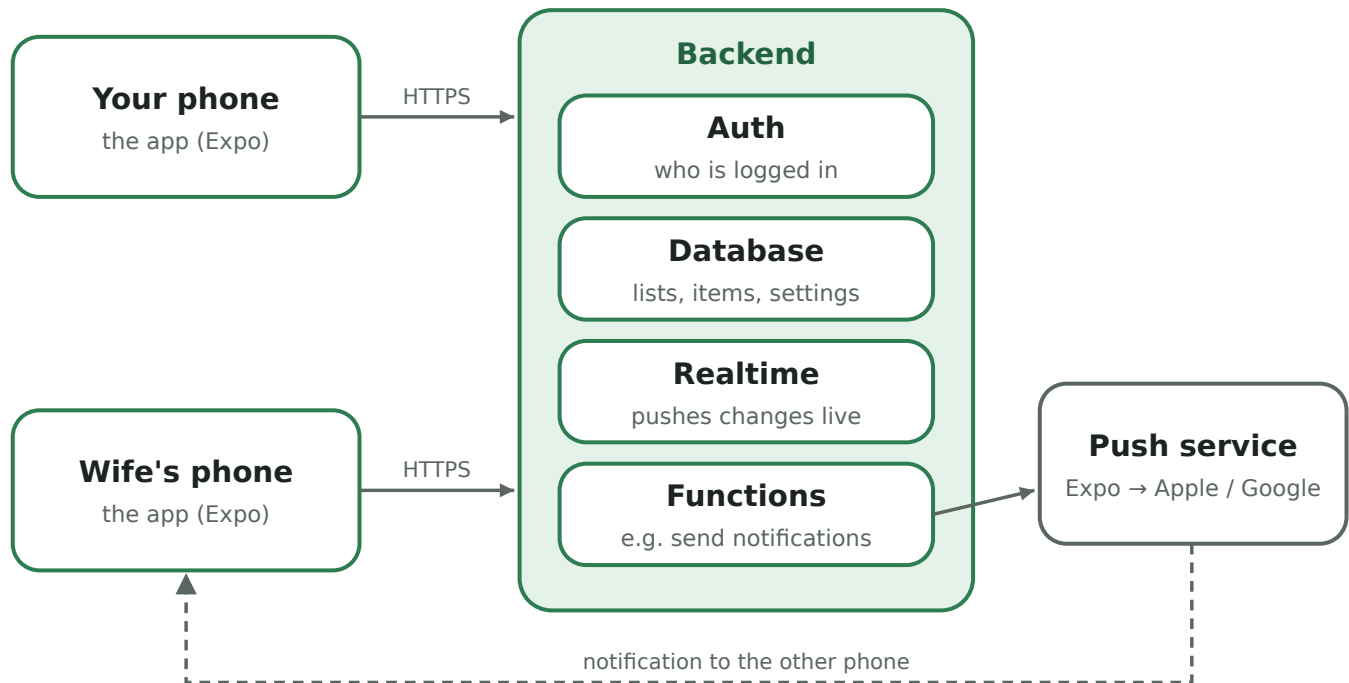
- **Sharing.** Your phone and your wife's phone each have their own copy of the data. Nothing you add shows up on the other phone.
- **Saving.** Everything resets when the app reloads.
- **Real accounts.** Login accepts any password for the two test emails.

The good news

The app was built so the mock can be swapped out. The rest of the app only talks to the functions in `api/`, and those functions already have the right names and types (`getLists`, `addItem`, `toggleItem` ...). Replacing their insides with real network calls is most of the work.

2. The big picture

A shopping list you share with someone needs a place on the internet where the lists live, so both phones read and write the same data. That place is the **backend**. Here is how all the pieces fit together:



There are five pieces to sort out. Each gets its own chapter:

Piece	What it does	Chapter
Backend	Stores the lists and items, checks who is logged in, and tells each phone when something changes.	3–5
Connecting the app	Replace the mock in <code>api/</code> with calls to the backend.	5
Push notifications	“Anna added Milk” on your lock screen, even when the app is closed.	6
Getting the app on your phones	Turning the code into an app you install, without needing Expo Go.	7
Hosting	Where the backend (and optionally a web version) runs, and what it costs.	8–9

3. What the backend has to do

It helps to be specific about what this app needs before choosing a service. Everything below already exists in the mock, so it's a checklist rather than a wish list.

3.1 Authentication: who are you?

Two accounts, one for each of you. Logging in returns a **session** (a token) that the app stores and sends with every request, so the backend knows who is asking. Today that's faked in `mock/api/auth.ts`.

- Email and password is enough. Sign-in with Google or Apple is possible later.
- After creating your two accounts, switch off new sign-ups, so nobody else can register.

3.2 Database: what is stored?

Four kinds of data, matching the types in `types/models.ts`:

Table	Contents
users / profiles	Name, email, and settings (language, appearance, which notifications you want).
lists	Name of each shopping list.
items	Name, amount, category, checked or not, which list, who added it, and when.
push_tokens	One row per phone, so the backend knows where to send notifications (chapter 6).

The data is **relational**: items belong to a list, and items are added by a user. That fits a classic SQL database (like PostgreSQL) very well.

3.3 API: how does the app talk to it?

The app needs to read and change data: get lists, add an item, check it off, delete checked items, and so on. These are the functions in `api/`. With a backend service (next chapter) you mostly don't write this API yourself, because the service generates it from your tables.

3.4 Realtime: seeing each other's changes

When your wife adds “Milk” at home, your list in the shop should update by itself. Today the app **polls** (asks “anything new?” every 10 seconds). A real backend can **push** changes to the app the moment they happen, over a connection that stays open (a WebSocket). That's faster and uses less battery.

3.5 Server-side logic: notifications

Some things must happen on the server, not in the app. The main one here: when someone adds or checks off an item, send a notification to the **other** person, if they have that notification switched on. That needs a small piece of code running on the backend (a function), described in chapter 6.

4. Choosing a backend

There are two broad ways to get a backend:

- **Backend-as-a-Service (BaaS).** A ready-made backend: database, logins, realtime and functions, all set up through a web dashboard. You define your tables and rules; the service gives you an API. Little code, fast to get going.
- **Your own server.** You write the API yourself (for example in Node.js) and run it with a database. Full control, but you build and maintain everything: logins, security, realtime, updates.

For a private app for two people, a BaaS is clearly the better fit. Here are the realistic options:

Option	Good	Less good	Cost
Supabase	PostgreSQL (SQL, fits your data). Auth, realtime and functions built in. Official Expo guide. Can be self-hosted later.	Free projects pause after a week without use (data kept; one click to wake up).	Free; Pro \$25/month
Firebase	Very mature, huge community, Google's push service (FCM) included.	NoSQL (Firestore), so the data model needs rethinking. Server functions require the pay-as-you-go plan.	Free tier; pay-as-you-go
PocketBase	One small program: database (SQLite), auth, realtime and admin UI. Open source (MIT), no usage limits. Never pauses.	You run it yourself on a server (chapter 8). You handle backups and updates.	Free software; server ~€4-5/month
Own Node.js API	Learn how everything works; total freedom.	Most work by far: logins, security, realtime and hosting are all on you.	Server ~€4-5/month

Recommendation: Supabase

Start with Supabase on the free plan. Your data is relational, so PostgreSQL fits naturally. Logins, realtime updates and server functions (for notifications) are all built in, it has an official guide for Expo apps, and it costs nothing for two users.

If the pausing bothers you (see the warning in chapter 8), or you'd rather own everything, PocketBase on a small server is the best alternative. The chapters below use Supabase as the example; the steps are the same in spirit for PocketBase.

5. Setting up Supabase and connecting the app

5.1 Create the project

1. Create an account at supabase.com and a new project. Choose a region close to you (for example Stockholm or Frankfurt).
2. Under Authentication, create the two users (you and your wife) with email and password.
3. Then switch off “Allow new users to sign up”, so the app is closed to everyone else.
4. Copy the project URL and the public (anon / publishable) key from the project settings. The app needs both.

5.2 Create the tables

Run this in the SQL editor. It mirrors types/models.ts. One deliberate change from the mock: items get a `created_at` timestamp, so the database can return them in the order they were added, which your sorting relies on.

```
create table profiles (
  id          uuid primary key references auth.users on delete cascade,
  name        text not null,
  settings    jsonb not null default '{"language":"sv","theme":"system",
    "notifications":{"itemAdded":false,"itemCompleted":false}}'
);

create table lists (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  created_at  timestampz not null default now()
);

create table items (
  id          uuid primary key default gen_random_uuid(),
  list_id     uuid not null references lists on delete cascade,
  name        text not null,
  amount      text,
  category    text,
  done        boolean not null default false,
  added_by    uuid references profiles,
  created_at  timestampz not null default now()
);

create table push_tokens (
  token       text primary key,
  user_id     uuid not null references profiles on delete cascade
);
```

on `delete cascade` on items means deleting a list also deletes its items, exactly like `deleteList` does in the mock.

5.3 Lock it down with Row Level Security

The public key is built into the app, so anyone could find it. What protects your data is **Row Level Security** (RLS): rules in the database that decide who may read or change each row. Since sign-ups are off, “logged in” means “one of the two of you”, which keeps the rules simple:

```

alter table profiles      enable row level security;
alter table lists         enable row level security;
alter table items         enable row level security;
alter table push_tokens   enable row level security;

-- Both of you share all lists and items.
create policy "household" on lists for all to authenticated using (true) with check (true);
create policy "household" on items for all to authenticated using (true) with check (true);

-- You can read both profiles (for names in notifications) but only change your own.
create policy "read profiles" on profiles for select to authenticated using (true);
create policy "own profile"   on profiles for update to authenticated using (id = auth.uid());

-- Each phone registers its own push token.
create policy "own tokens" on push_tokens for all to authenticated
  using (user_id = auth.uid()) with check (user_id = auth.uid());

```

Never skip RLS

A table without RLS enabled can be read and changed by anyone who has your public key. Supabase shows a warning in the dashboard for such tables. Treat it as a hard rule: every table gets RLS.

5.4 Connect the app

Install the Supabase client, plus the storage it uses to remember the login on the phone:

```
npx expo install @supabase/supabase-js @react-native-async-storage/async-storage
```

Put the URL and key in a file called `.env` in the project root. Expo makes variables starting with `EXPO_PUBLIC_` available in the app. Your `.gitignore` already excludes `.env*.local`, so name it `.env.local`:

```

EXPO_PUBLIC_SUPABASE_URL=https://<your-project>.supabase.co
EXPO_PUBLIC_SUPABASE_KEY=<public anon / publishable key>

```

Create one shared client, for example in `api/supabase.ts`. Follow Supabase's “Expo React Native” quickstart for the exact current setup; it looks roughly like this:

```

import AsyncStorage from "@react-native-async-storage/async-storage";
import { createClient } from "@supabase/supabase-js";

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_KEY!,
  { auth: { storage: AsyncStorage, autoRefreshToken: true, persistSession: true } },
);

```

5.5 Replace the mock, one api/ file at a time

Each file in `api/` currently says `export * from "@mock/api/items"`. Replace that line with real implementations that keep the same function names and return the same types. For example, `api/items.ts`:


```
import { supabase } from "@api/supabase";
import { Item } from "@types/models";

// The database uses snake_case columns; the app uses camelCase.
const toItem = (row: any): Item => ({
  id: row.id, listId: row.list_id, name: row.name, done: row.done,
  addedBy: row.added_by, amount: row.amount ?? undefined, category: row.category ?? undefined,
});

export async function getItem(listId: string): Promise<Item> {
  const { data, error } = await supabase
    .from("items").select("*").eq("list_id", listId)
    .order("created_at"); // order of adding
  if (error) throw error;
  return data.map(toItem);
}

export async function toggleItem(itemId: string, done: boolean): Promise<void> {
  const { error } = await supabase.from("items").update({ done }).eq("id", itemId);
  if (error) throw error;
}
```

Work through the files in this order, testing the app after each one:

File	Notes
auth.ts	Use <code>supabase.auth.signInWithPassword</code> and <code>signOut</code> . Supabase stores the session itself, so <code>AuthContext</code> no longer needs <code>helpers/storage.ts</code> for the token. Load the profile (name, settings) after login.
settings.ts	Update the settings column of your profile row.
users.ts	Select id and name from profiles.
lists.ts	Select, insert and delete on lists. Deleting cascades to items.
items.ts	As above. Small signature changes (like <code>toggleItem</code> taking the new value) are fine: update <code>useItems</code> to match.
activity.ts	No longer needed: <code>realtime</code> (5.6) and <code>push</code> (6) replace polling. Remove <code>useActivityNotifications</code> ' polling and the simulator.

5.6 Turn on realtime

In the dashboard, enable realtime for the lists and items tables. Then, in one place (for example `ListsContext`), subscribe to changes and tell `TanStack Query` to refetch. The screens update by themselves:

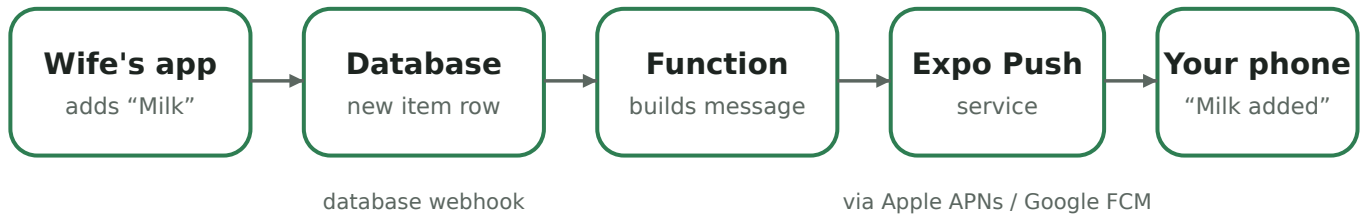
```
useEffect(() => {
  const channel = supabase
    .channel("household")
    .on("postgres_changes", { event: "*", schema: "public", table: "items" },
      () => queryClient.invalidateQueries({ queryKey: ["items"] }))
    .on("postgres_changes", { event: "*", schema: "public", table: "lists" },
      () => queryClient.invalidateQueries({ queryKey: ["lists"] }))
    .subscribe();
  return () => { supabase.removeChannel(channel); };
}, []);
```

5.7 Remove the mock

When every api/ file uses Supabase and the app works on both phones: delete the mock/ folder, the startMockActivity effect in hooks/useActivityNotifications.ts, and the test-account hint on the login screen. The README's "Connecting a real backend" section lists the same steps.

6. Push notifications

Right now notifications are created by the app itself while it's open. Real push notifications arrive even when the app is closed, so they have to be sent from the backend. The chain looks like this:



6.1 How push works

- **Push token.** When the app starts, it asks the phone for a push token: an address that identifies this app on this phone. The app saves it in the `push_tokens` table.
- **Expo Push Service.** Expo runs a free service that accepts a message plus a token, and forwards it to Apple (APNs) or Google (FCM). You don't need to talk to Apple or Google directly.
- **Trigger.** A database webhook calls a Supabase Edge Function whenever a row in `items` is inserted or updated.
- **Function.** It works out who did what, looks up the other person's settings and tokens, and sends the message.

6.2 The pieces to build

1. In the app: get the token with `Notifications.getExpoPushTokenAsync({ projectId })` after login, and insert it into `push_tokens`. The `projectId` comes from EAS (chapter 7).
2. Write an Edge Function (TypeScript, running on Supabase) that receives the changed item, skips the person who made the change, checks their notification settings, and sends to Expo's push API.
3. Create a database webhook on the `items` table (insert and update) that calls the function.
4. Remove the in-app notification code that is no longer needed (the polling in `useActivityNotifications`).

The core of the function is a single HTTP request:

```

await fetch("https://exp.host/--/api/v2/push/send", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(
    tokens.map((to) => ({ to, title: listName, body: `${who} added ${item.name}` })),
  ),
});

```

Push needs a real build, not Expo Go

Since Expo SDK 53, remote push notifications don't work in Expo Go on Android, so you need a development or preview build (chapter 7) to test them. On iPhone, push also requires the paid Apple Developer Program. Local notifications (what the app does today) still work in Expo Go.

7. Getting the app onto your phones

Until now you've run the app through Expo Go or the browser. To use it day to day, you build it into a real app with its own icon. Expo's build service, **EAS Build**, does this in the cloud, so you don't need Android Studio or a Mac for building.

7.1 The kinds of builds

Build	What it is	Use it for
Expo Go	Expo's own app that loads your code. No install needed, but limited (no remote push, no own icon).	Quick development.
Development build	Your own app with developer tools inside. Loads code live from your computer like Expo Go.	Developing with push and native features.
Preview build	Your finished app as an installable file, shared directly. No store involved.	Using it yourselves: this is the one you want.
Store build	Uploaded to Google Play / App Store.	Not needed for two people.

7.2 One-time setup

```
npm install -g eas-cli
eas login          # free Expo account
eas init           # links the project, adds the projectId to app.json
eas build:configure # creates eas.json
```

In eas.json, make the preview profile produce a normal installable APK file for Android:

```
"preview": {
  "distribution": "internal",
  "android": { "buildType": "apk" }
}
```

7.3 Android

1. Run `eas build --profile preview --platform android`. It builds in the cloud and gives you a download link and QR code.
2. Open the link on each Android phone and install. The phone will ask you to allow installing apps from that source.
3. For push on Android, EAS needs Firebase Cloud Messaging credentials. The EAS docs walk you through creating a free Firebase project for this (only for push delivery, not as your backend).

Cost: nothing. No Google Play account is needed to install your own APK.

7.4 iPhone

Apple doesn't allow installing apps freely. You need the **Apple Developer Program** (\$99/year). Then

you have two ways in:

- **Ad hoc (internal) build:** register each iPhone with `eas device:create`, then `eas build --profile preview --platform ios` and install from the link. Good for exactly your two phones.
- **TestFlight:** build for the store profile and run `eas submit`; you then install through Apple's TestFlight app without publishing publicly. Builds expire after 90 days, so you rebuild now and then.

7.5 Updating the app later

Most changes are JavaScript only (screens, logic, styles). Those can be sent to the installed app with **EAS Update** (`eas update`), without building and reinstalling. You only need a new build when you add a package with native code (like `react-native-svg` or `expo-notifications`) or change `app.json`.

7.6 Alternative: the web version

The app also runs in a browser. `npx expo export -p web` produces a static website you can host for free (chapter 8) and open on both phones, even adding it to the home screen. It's the cheapest way to include an iPhone. The catch: push notifications on the web need extra work (a service worker and web push), and on iPhone they only work when the site is added to the home screen. Native builds are the smoother path for notifications.

8. Hosting: where things run

“Hosting” means the computer on the internet where your backend runs. Here are concrete examples, from easiest to most hands-on:

Service	What you'd run there	Price (Oct 2026)
Supabase Cloud	The whole backend: database, auth, realtime, functions. Nothing to install or update.	Free: 2 projects, 500 MB database, 50,000 users. Pro: \$25/month.
Firebase	Firestore database, auth, push. Server functions need the pay-as-you-go plan.	Spark (free): 50K reads + 20K writes per day, 1 GiB.
Hetzner Cloud (VPS)	A small Linux server for PocketBase or your own Node API. You install and update it yourself.	CX22 (2 CPU, 4 GB RAM): about €4.50/month.
Railway / Fly.io	Platforms that run PocketBase or a Node API from a template, without managing a server.	Usage-based, typically a few dollars a month for something this small.
Raspberry Pi at home	PocketBase on a small computer in your home. Use Cloudflare Tunnel (free) for a secure HTTPS address without opening your router.	Hardware once, then only electricity.
Vercel / Netlify / Cloudflare Pages	Only the web version of the app (static files from expo export).	Free hobby plans are plenty.
EAS (Expo)	Builds the Android and iOS apps, sends app updates.	Free: 15 Android + 15 iOS builds per month.

Supabase free plan: pausing

Free Supabase projects pause after one week without any activity. Your data is kept, but the app stops working until you click “Restore” in the dashboard. A shopping list you use several times a week will normally stay awake. If it does pause during a holiday, restoring takes a minute. If that's not acceptable, upgrade to Pro (\$25/month) or use PocketBase on a VPS (about €4.50/month), which never pauses.

8.1 What “self-hosting” involves

If you choose PocketBase (or your own API) on a VPS, you take on some server care. None of it is hard, but it's yours to do:

- Install it once, and set it up to start automatically (a systemd service).
- Put it behind HTTPS (for example with Caddy, which gets certificates automatically).
- Back up the database file regularly, ideally to somewhere other than the server.
- Install system and PocketBase updates now and then.

9. What it costs

Three realistic setups, per year:

Setup	What you pay for	Per year
Both on Android + Supabase free	Nothing: free backend, free builds, APK installed directly.	0 kr
At least one iPhone + Supabase free	Apple Developer Program, needed to install on an iPhone and for iOS push.	\$99
Android + PocketBase on Hetzner	The VPS. Add \$99 if an iPhone is involved.	≈ €54
Any of the above, never-pausing Supabase	Supabase Pro instead of free.	+ \$300

Cheapest way to include an iPhone

Use the web version (chapter 7.6) on the iPhone and the native app on Android. It costs nothing, but notifications on the iPhone need the extra web-push work, or have to wait.

10. Step-by-step roadmap

Each phase leaves you with something that works, so you can stop at any point and still use the app.

Phase 1: Backend (one evening)

- Create the Supabase project, the two users, and switch off sign-ups.
- Create the tables and RLS policies (chapter 5.2–5.3).
- Add a list and a few items by hand in the dashboard's table editor to see it working.

Phase 2: Connect the app

- Install the client and add `.env.local` (5.4).
- Replace `api/auth.ts` first and log in with a real account.
- Replace lists, items, settings and users. Test after each one.
- Check it on two devices (for example browser + emulator) logged in as different users.

Phase 3: Live updates

- Enable realtime and add the subscription (5.6). Remove polling. Delete mock/ (5.7).

Phase 4: Install on your phones

- Set up EAS and make a preview build (chapter 7). Install it on both phones. From here on, use the real app.

Phase 5: Push notifications

- Register push tokens, write the Edge Function, add the webhook (chapter 6). Make a new build, since expo-notifications needs native code.

Phase 6: Nice to have

- **Offline support:** the shop basement often has no signal. TanStack Query can keep the last data and queue changes.
- **Backups:** export the database now and then (Supabase free has no automatic backups you can restore yourself).
- **Your own app icon in the build:** already done; it appears once you install a real build.

11. Security checklist

Check



Sign-ups are switched off after creating your two accounts.

Every table has Row Level Security enabled, with policies.

Only the public (anon / publishable) key is in the app. The service role / secret key is never in the app or in git; only Edge Functions use it.

.env.local is not committed (your .gitignore already covers it).

The backend is reached over HTTPS only (Supabase does this for you).

You have a recent export of the database somewhere safe.

12. Glossary

Term	Meaning
API	The set of operations an app can ask a backend to do, like “get items for this list”.
BaaS	Backend-as-a-Service: a ready-made backend (Supabase, Firebase, PocketBase).
Edge Function	A small piece of server code that runs on Supabase when called, for example by a webhook.
EAS	Expo Application Services: builds, submits and updates your app.
Expo Go	Expo's app for running your code during development without building it.
Push token	An address that identifies your app on one specific phone, used to send it notifications.
APNs / FCM	Apple's and Google's push notification services.
Realtime	The backend pushes changes to the app as they happen, instead of the app asking repeatedly.
RLS	Row Level Security: database rules that decide who may read or change which rows.
Session	Proof that you're logged in, stored by the app and sent with each request.
VPS	Virtual Private Server: a small rented Linux computer on the internet.
Webhook	An automatic HTTP call made when something happens, like a new row in a table.

Sources

Prices and limits were checked in October 2026 and change over time. Check the providers' own pages before you rely on them.

- Supabase pricing and free-tier limits: supabase.com/pricing; summarised at jetadmin.io and automationatlas.io (2026).
- Expo EAS plans: docs.expo.dev/billing/plans; expo.dev/pricing.
- expo-notifications and Expo Go push support: docs.expo.dev/versions/v54.0.0/sdk/notifications.
- Apple Developer Program (\$99/year) and Google Play (\$25 one-time): developer.apple.com; play.google.com/console.
- Hetzner Cloud CX22 pricing: hetzner.com/cloud; vpsbenchmarks.com/hosters/hetzner/plans/cx22.
- Firebase Spark limits: firebase.google.com/pricing.
- PocketBase: pocketbase.io.